

BINARY SEARCH

THE BINARY SEARCH EXPLAINED

- The binary search algorithm can be performed on sorted arrays, usually in ascending order.
- The first iteration of the array tests the middle element of the array.
- If this matches the search key the algorithm ends.
- If the search key is less than the middle element the algorithm then searches the first half of the array (the first element up to but not including the middle element)
- If the search key is greater than the middle element the algorithm searches the second half of the array (the element after the middle element to the last element)
- With each iteration the algorithm tests the middle element in the remaining portion of the array and eliminates half of the array.
- The algorithm ends by finding the search key or reducing the sub array to a size of zero.

BINARY SEARCH USES

- A binary search can be used to:
 - look up a word in a dictionary
 - find a library book
 - look through a list of contacts
 - search an ordered list

EXAMPLE

2	5	9	13	21	33	46	52	66
---	---	---	----	----	----	----	----	----

Search Key

9

- The binary search algorithm first checks 21 because 21 is the middle element of the array.
- 21 is not a match.
- The algorithm checks if the search key is greater or less than 21. It is less than.
- The algorithm ignores 21 and all the elements greater than 21.

2	5	9	13
---	---	---	----

- The algorithm then checks 9. It is a match.
- The algorithm returns the index 2 which is the location of 9 in the array.

ANOTHER EXAMPLE

4	5	10	11	16	22	31	45	58
---	---	----	----	----	----	----	----	----

Search Key

45

- The binary search algorithm first checks 16 because 16 is the middle element of the array.
- 16 is not a match.
- The algorithm checks if the search key is greater or less than 16. It is greater.
- The algorithm ignores 16 and all the elements less than 16.

22	31	45	58
----	----	----	----

- The algorithm then checks 45. It is a match.
- The algorithm returns the index 7 which is the location of 45 in the array.

BINARY SEARCH IMPLEMENTATION

```
public static int binarySearch(int[] data, int key){  
    int low = 0;  
    int high = data.length - 1;  
    int middle = ((low + high) + 1)/2;  
    int location = -1;  
    do{  
        if(key == data[middle])  
            location = middle;  
        else if(key < data[middle])  
            high = middle-1;  
        else  
            low = middle+1;  
        middle = ((low + high) + 1)/2;  
    }while((low <= high) && (location == -1));  
    return location;  
}
```

BINARY SEARCH IMPLEMENTATION

- The `binarySearch` method performs the binary search.
- Two parameters - the array to be searched and the search key.
- Initial values are set for, low, high and middle based on length of the array. The location variable is initialised to -1.
- A loop is used to traverse the array as required.
- The search key is compared to the middle element of the array. If it is a match the index is returned.
- If the search key is less than the middle element, the algorithm ignores the second half of the array, high is given a value of middle - 1.
- If the search key is greater than the middle element, the algorithm ignores the first half of the array, low is given a value of middle + 1.
- Each iteration tests a single value and excludes half of the remaining values in the array from the search space.

BINARY SEARCH IMPLEMENTATION

- The remainingElements method creates a string of the array elements in the search space so it can be displayed
- Three parameters - the array, the current low and the current high.
- An instance of the StringBuilder class is used.
- A loop is used to traverse from the current low to the current high.
- Each element of the array is appended to the string.
- The string is returned.

```
public static String remainingElements(int[] data, int low, int high){  
    StringBuilder temp = new StringBuilder();  
  
    for(int i=low; i<=high; i++)  
        temp.append(data[i] + " ");  
    return String.format("%s\n", temp);  
}
```